

Data Structures and Algorithms

Lab 04 - Applications of Linked Lists

Instructor: Sidrat-ul-Muntha

READ IT FIRST

Prior to start solving the problems in this assignments, please give full concentration on following points.

1. **WORKING** – This is individual lab. If you are stuck in a problem contact your teacher, but, in mean time start doing next question (don't waste time).
 2. **DEADLINE** – One Week after assigned this.
 3. **SUBMISSION** – This assignment needs to be submitted in a soft copy.
 4. **WHERE TO SUBMIT** – Please visit your LMS.
 5. **HOW TO SUBMIT** – Rename all .java files according to the task number (e.g., Task1.java, Task2.java). Place all required .java files in a .zip file named 000-00-0000_assignment.zip and submit the ZIP file on the LMS.
-

Task 1: Shopping Cart Using Singly Linked List

Create a **Shopping Cart Management System** using a singly linked list. Each node should store a custom **Product** object.

Product Class

Each **Product** should contain:

- productId
- productName
- price
- quantity

Required Methods

- **addProductFront(Product product)** – Add a product at the beginning of the cart.
- **addProductBack(Product product)** – Add a product at the end of the cart. Must use the **tail** pointer.

- **removeProduct(int productId)** – Remove a product using its product ID. Correctly update **head** and **tail**.
- **searchProduct(int productId)** – Search for a product by ID. Return the product if found; otherwise return null.
- **updateQuantity(int productId, int quantity)** – Find the **product** and update its **quantity**.
- **calculateTotal()** – Calculate the total cost of all products using **price × quantity**.
- **displayCart()** – Display **all products** from head to tail.
- **size()** – Return the **number of products** in the cart.
- **clearCart()** – Remove all products from the cart. After clearing the cart, correctly set both **head** and **tail** to null and reset the **size** to 0.
- **calculateTotalItems()** – Calculate and return the total number of individual items in the cart by adding the **quantity** of every product.
- **increaseQuantity(int productId, int amount)** – Find the product using its **product ID** and increase its quantity by the specified **amount**.
- **decreaseQuantity(int productId, int amount)** – Find the product using its **product ID** and decrease its quantity by the specified **amount**. The quantity must not become negative.
- **findMostExpensiveProduct()** – Search through the cart and return the product with the **highest price**. Return null if the cart is empty.
- **calculateProductCost(int productId)** – Find the product using its **product ID** and calculate its total cost using **price × quantity**. Return the calculated cost if the product is found.

Important Requirements

- You must use a **singly linked list**.
- You must use a custom **Product object**.
- The linked list must maintain both **head** and **tail** pointers.
- **addProductBack()** must use the **tail** pointer and have **O(1)** time complexity.
- **removeProduct()** and **searchProduct()** must have **O(n)** time complexity.
- **calculateTotal()** must have **O(n)** time complexity.
- Correctly maintain **head** and **tail** after every insertion and deletion.
- If the cart becomes empty, both **head** and **tail** must be null.
- Do not use Java's built-in **LinkedList** class

Example

[101, Apple, 250, Qty: 3] --> [102, Banana, 180, Qty: 5] --> [103, Tomato, 200, Qty: 2] --> [104, Potato, 150, Qty: 4] --> null

Task 2: To-Do List Using Singly Linked List

Create a **To-Do List Management System** using a **singly linked list**. Each node should store a custom **Task** object.

Task Class

Each **Task** should contain:

- taskId
- title
- description
- status ("Pending" or "Completed")

Required Methods

- **addTaskFront(Task task)** – Add a task at the beginning of the list.
- **addTaskBack(Task task)** – Add a task at the end of the list. This method must use the tail pointer.
- **removeTask(int taskId)** – Remove a task using its ID. Correctly update the head and tail pointers.
- **searchTask(int taskId)** – Search for a task by ID. Return the task if found; otherwise, return null.
- **markCompleted(int taskId)** – Find the task by ID and change its status to "Completed".
- **displayPendingTasks()** – Display only tasks whose status is "Pending".
- **displayAllTasks()** – Display all tasks from head to tail.
- **size()** – Return the total number of tasks in the list.
- **markPending(int taskId)** – Find the task using its **task ID** and change its status to "Pending".
- **displayCompletedTasks()** – Display only the tasks whose status is "**Completed**", from head to tail.
- **countPendingTasks()** – Count and return the total number of tasks whose status is "Pending".
- **countCompletedTasks()** – Count and return the total number of tasks whose status is "Completed".

Important Requirements

- You must use a **singly linked list**.
- You must use a **custom Task object** rather than storing primitive data directly in the linked list.
- The linked list must maintain both **head and tail pointers**.
- **addTaskBack()** must use the **tail pointer** and have **O(1)** time complexity.
- **removeTask()** and **searchTask()** should have **O(n)** time complexity.
- **displayPendingTasks()** should have **O(n)** time complexity.
- Correctly maintain the **head and tail pointers** after every insertion and deletion.
- If the list becomes empty, both **head and tail must be null**.
- Do not use Java's built-in **LinkedList** class.

Example

[1, Lab 1, Complete Lab 1, Pending] --> [2, Lab 2, Complete Lab 2, Completed] -->

[3, Lab 3, Complete Lab 3, Pending] --> [4, Assignment 1, Complete Assignment 1, Pending] -->

[5, Assignment 2, Complete Assignment 2, Completed] --> null
